Stock Price Trend Prediction

Machine Learning & Cloud-Based Predictive Analytics Prototype

Date: March 2026

Course: SWE-452

Instructor: Professor Woodward

Team Members:
- Nathan Kennedy
- Batyr Rasulov
- Alejandro Barranco-Leyte

video link: https://youtu.be/Z09z7nIk5RM

Table of Contents

1. Project Overview
2. Requirements
3. Implementation Plan
4. Risk Assessment
5. Test & Validation Plan
6. Appendices

1. Project Overview

Objective

The objective of this project is to develop a machine learning-based system capable of predicting whether a stock's closing price will move upward or downward on the next trading day.

Unlike traditional approaches that attempt to forecast exact stock prices, this system focuses on trend classification, which provides more realistic and interpretable results given market volatility.

The project aims to:

- Achieve 60–65% prediction accuracy
- Maintain an F1 Score ≥ 0.58
- Demonstrate real-world application of machine learning and cloud deployment (MLOps concepts)

Scope

Included

- Historical stock data collection using Yahoo Finance
- Data preprocessing and cleaning
- Feature engineering (moving averages, returns, volatility)
- Machine learning model development (Logistic Regression, Random Forest)
- Model evaluation (accuracy, precision, recall, F1-score)
- Visualization of predicted vs actual trends
- Cloud deployment using AWS EC2 and S3
- REST API for predictions
- Streamlit dashboard for user interaction

Excluded

- Exact stock price prediction
- Real-time trading or financial automation
- High-frequency trading systems
- Financial decision-making guarantees

Stakeholders

Nathan Kennedy – ML Engineer
Batyr Rasulov – Cloud Engineer
Alejandro Barranco-Leyte – Data Engineer

2. Requirements

Updated Requirements

The following updates reflect prototype implementation decisions:

- API response latency must be **≤ 200ms**
- Deployment will use **AWS EC2 for compute and S3 for storage**
- Monitoring will be implemented using **AWS CloudWatch**

Dataset specifications:

- 5–10 years of historical stock data
- Approximately 2,500 records per stock

Binary classification labeling:

- **1 → Price increases**
- **0 → Price decreases**

3. Implementation Plan

System Workflow

The system follows a structured machine learning pipeline:

Data Collection → Preprocessing → Feature Engineering → Model Development → Evaluation → Deployment → Visualization

Development Steps

1. Data Collection

- Source: Yahoo Finance API via yFinance
- Data includes: Open, High, Low, Close, Volume

2. Data Preprocessing

- Handle missing values (forward fill or removal)
- Normalize data using scaling techniques
- Maintain chronological order using time-series split

3. Feature Engineering

- Moving averages (MA10, MA50)
- Daily returns
- Volatility

4. Model Development

- Logistic Regression
- Random Forest
- Use time-series-aware train/test split to prevent data leakage

5. Model Evaluation

- Accuracy
- Precision and Recall

- F1 Score
- Confusion Matrix

Target Metrics:

- Accuracy ≥ 60%
- F1 Score ≥ 0.58

6. Cloud Infrastructure Setup

- AWS EC2 instance (Ubuntu)
- Model storage in AWS S3
- IAM roles for secure access

7. API Development

- REST endpoint: /predict
- JSON request/response format
- Secure communication via HTTPS

8. Visualization (Frontend)

- Streamlit dashboard with:
    - Stock ticker input
    - Prediction output
    - Confidence probability
    - Historical trend visualization

9. Monitoring & Logging

- AWS CloudWatch logging
- Performance tracking
- Periodic evaluation

4. Risk Assessment

Identified Risks and Mitigation Strategies

Risk	Impact	Mitigation
Market Noise	Reduced accuracy	Focus on trend prediction instead of exact price
Overfitting	Poor generalization	Use time-series split and limit model complexity
Data Quality Issues	Biased predictions	Clean and validate datasets
Unrealistic Expectations	Misinterpretation	Provide disclaimers and explain limitations
Cloud Failure	System downtime	Maintain local backups and redundancy
API Latency	Slow response	Optimize model and infrastructure

5. Test & Validation Plan

Testing Objectives

- Validate model performance
- Ensure system reliability
- Verify API functionality
- Confirm cloud deployment

Testing Types

1. Model Testing

- Evaluate accuracy, precision, recall, and F1 score
- Compare multiple models

2. API Testing

- Send JSON requests to /predict endpoint
- Validate output correctness
- Ensure response time < 200ms

3. Integration Testing

- Validate full pipeline:
  Data → Model → API → Dashboard

4. Performance Testing

- API response time < 200ms
- Dashboard load time < 3 seconds

Test Data

- Source: Yahoo Finance
- Dataset:
  - 5–10 years of historical data
  - ~2,500 records per stock

Success Criteria (Proof of Concept)

The prototype is considered successful if:

- Accuracy ≥ 60%
- F1 Score ≥ 0.58
- API response time < 200ms
- Dashboard fully functional
- Cloud deployment operational

6. Appendices

Appendix A: Sprint 2 Document
Appendix B: Sprint 1 Risk Assessment